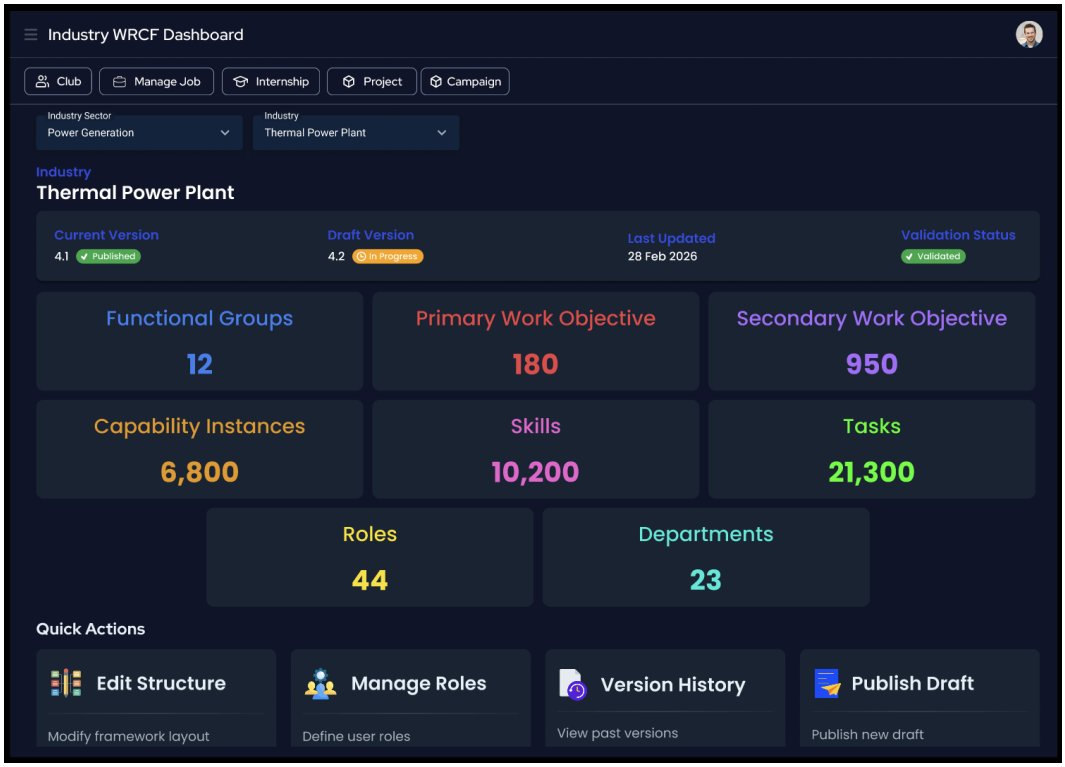
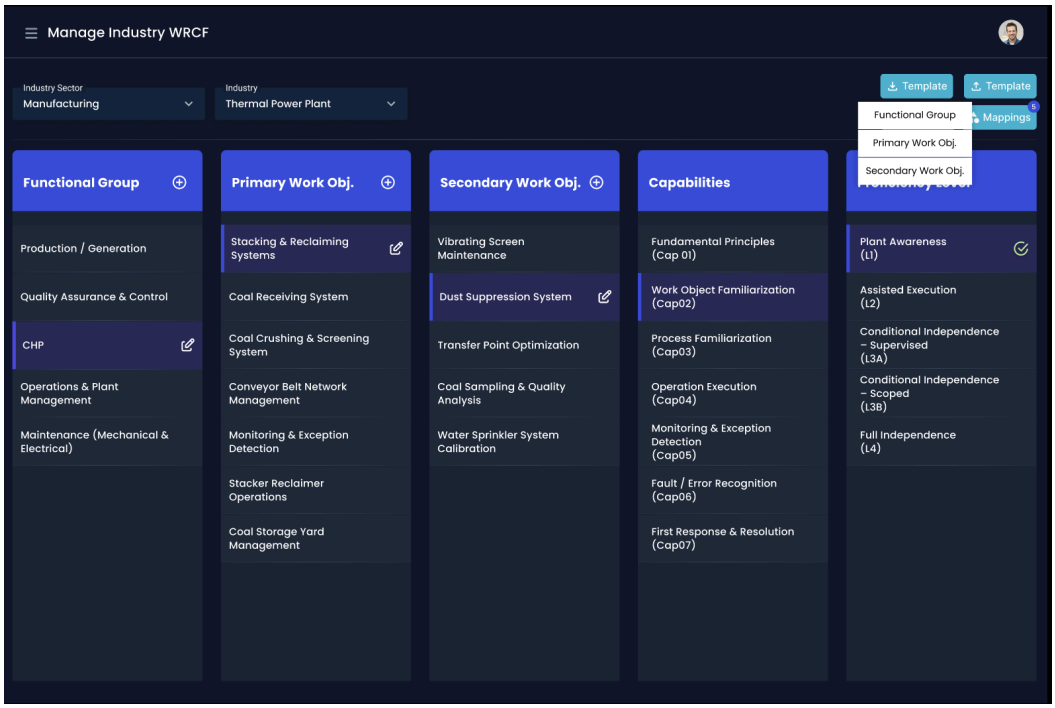
WRCF Module
Functional Use Case Document
with Detailed User Stories

Based on UI screenshot analysis and the stated CRUD scope for WRCF business entities

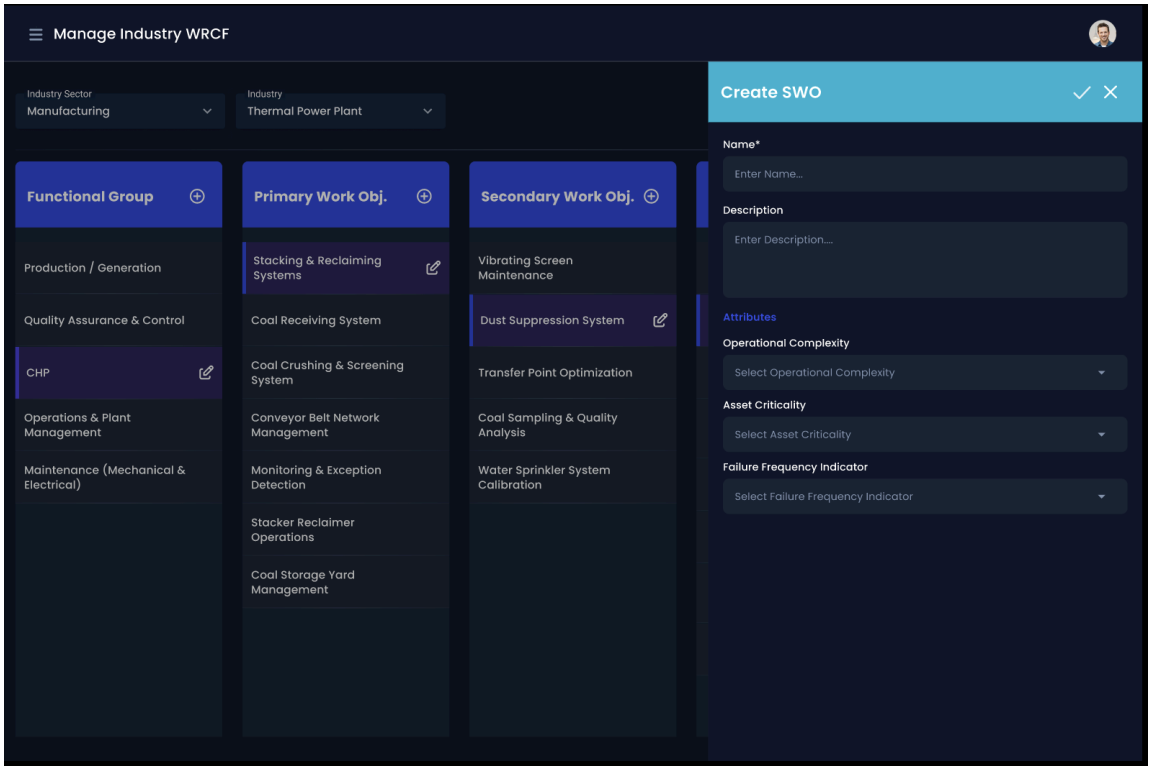
Document Type	Prepared For	Scope Basis
Functional specification	WRCF module analysis	Ten UI screenshots + user clarification



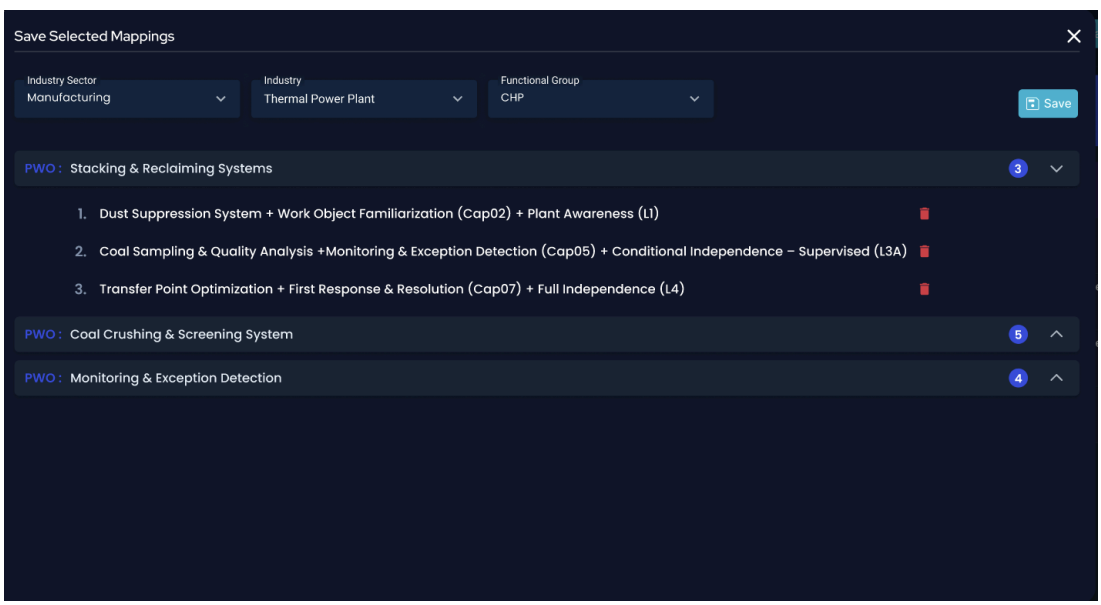
Picture 1: WRCF Dashboard



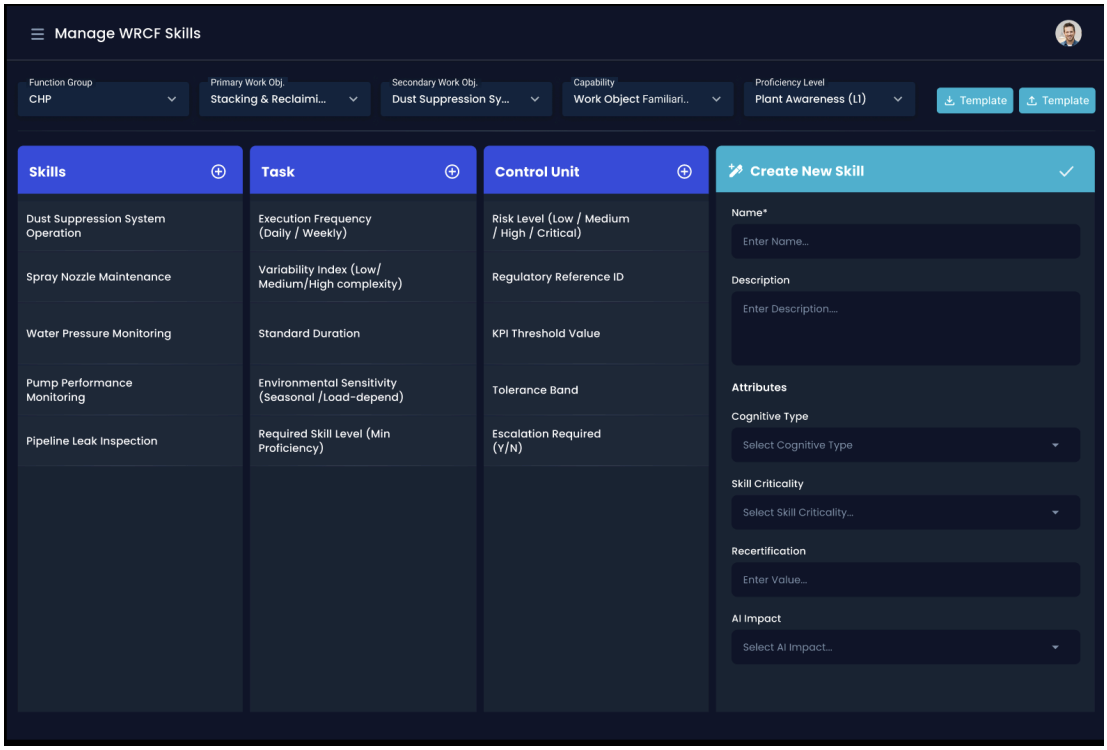
Picture 2: Manage WRCF



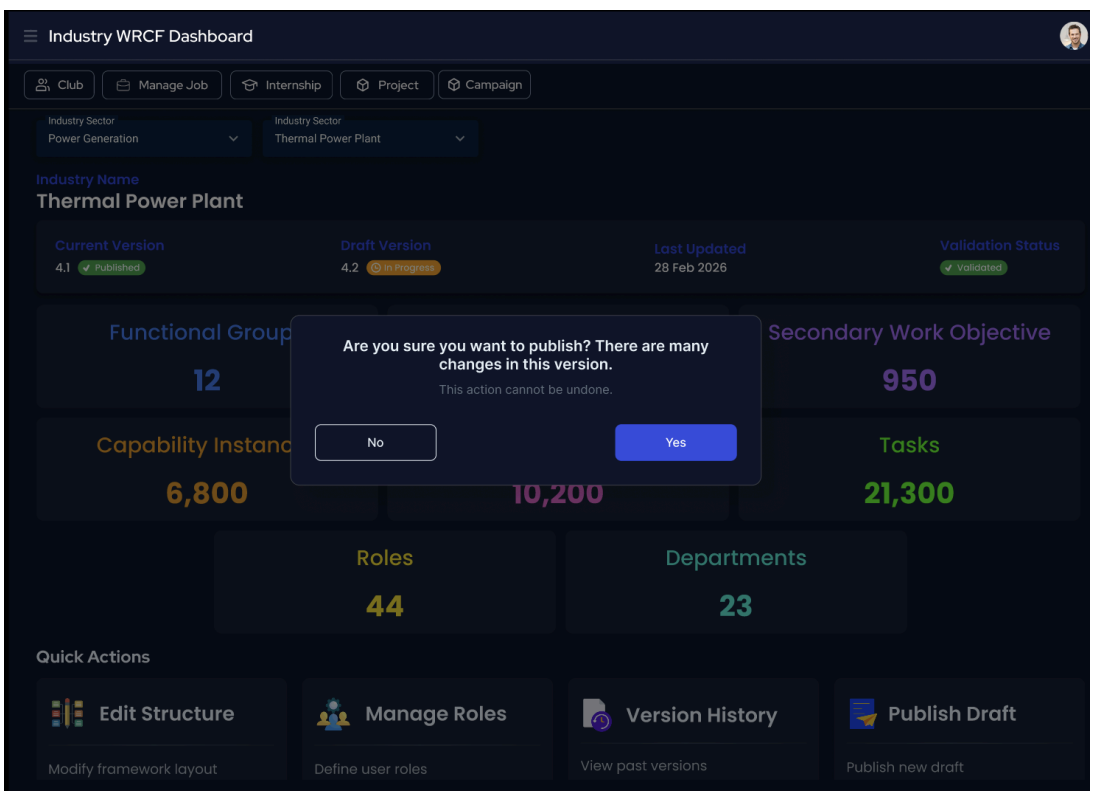
Picture 3: Create SWO



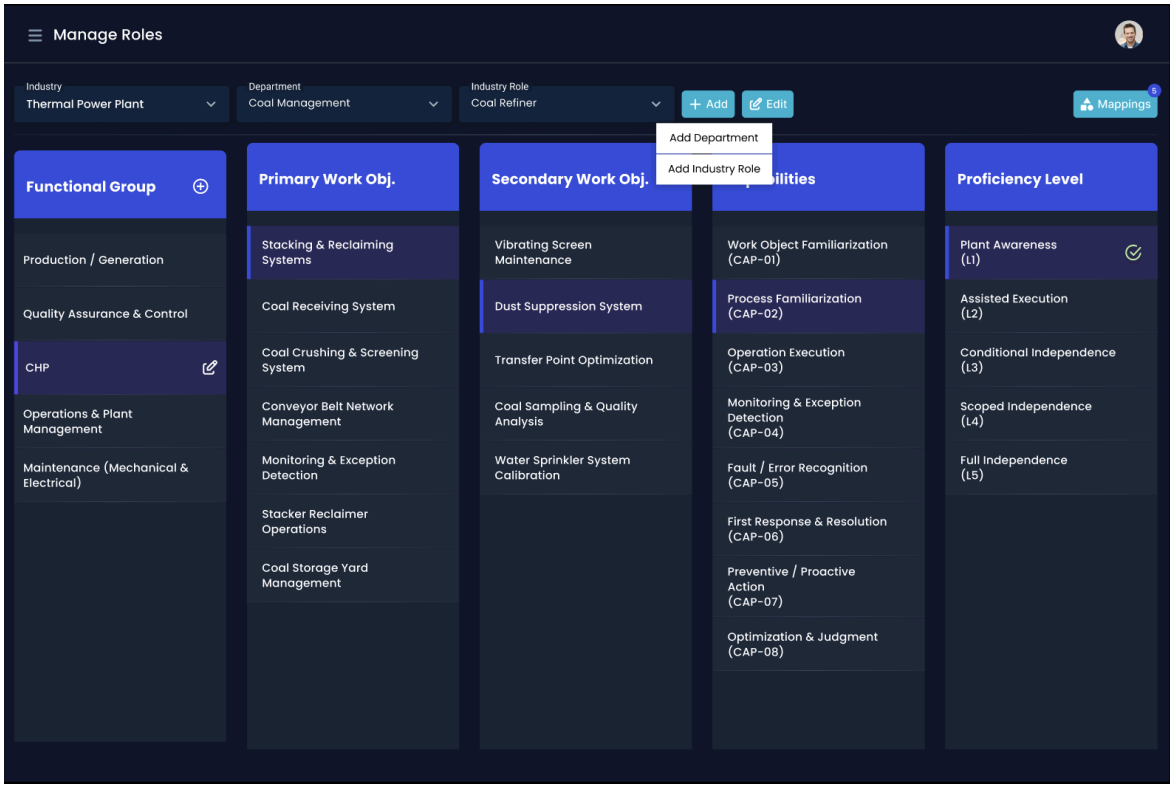
Picture 4: Save Capability Instances



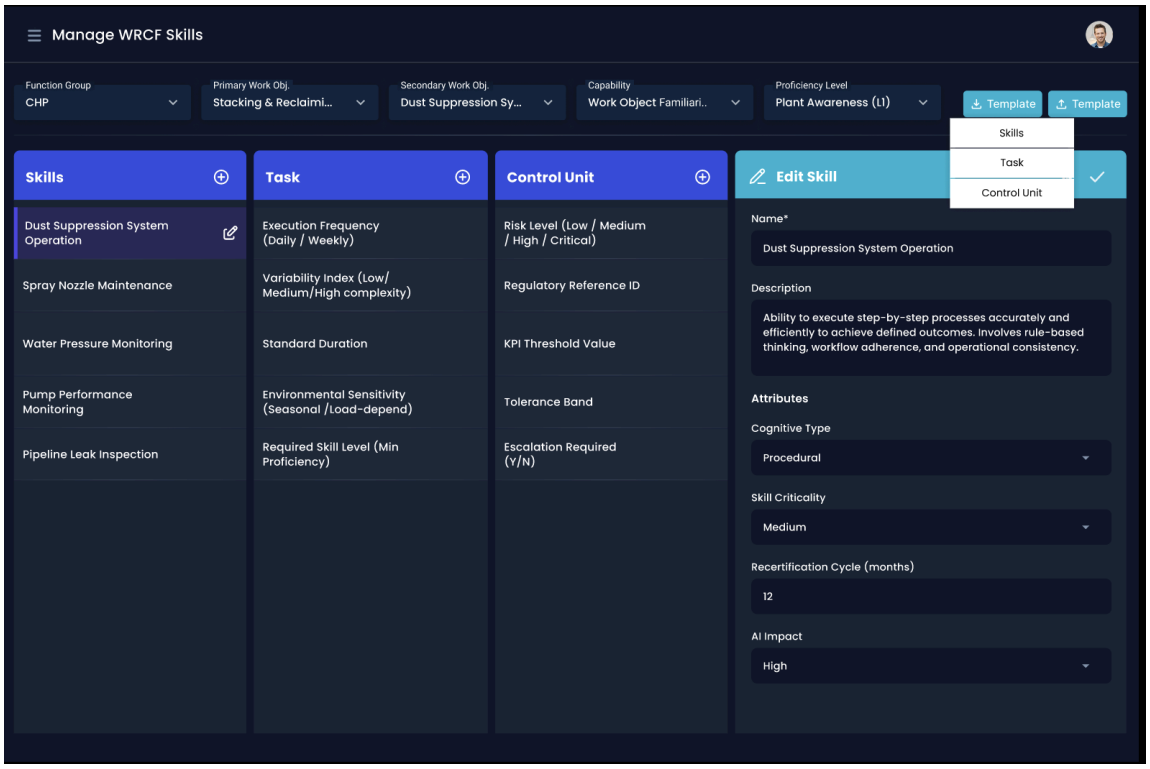
Picture 5: Manage WRCF Skills



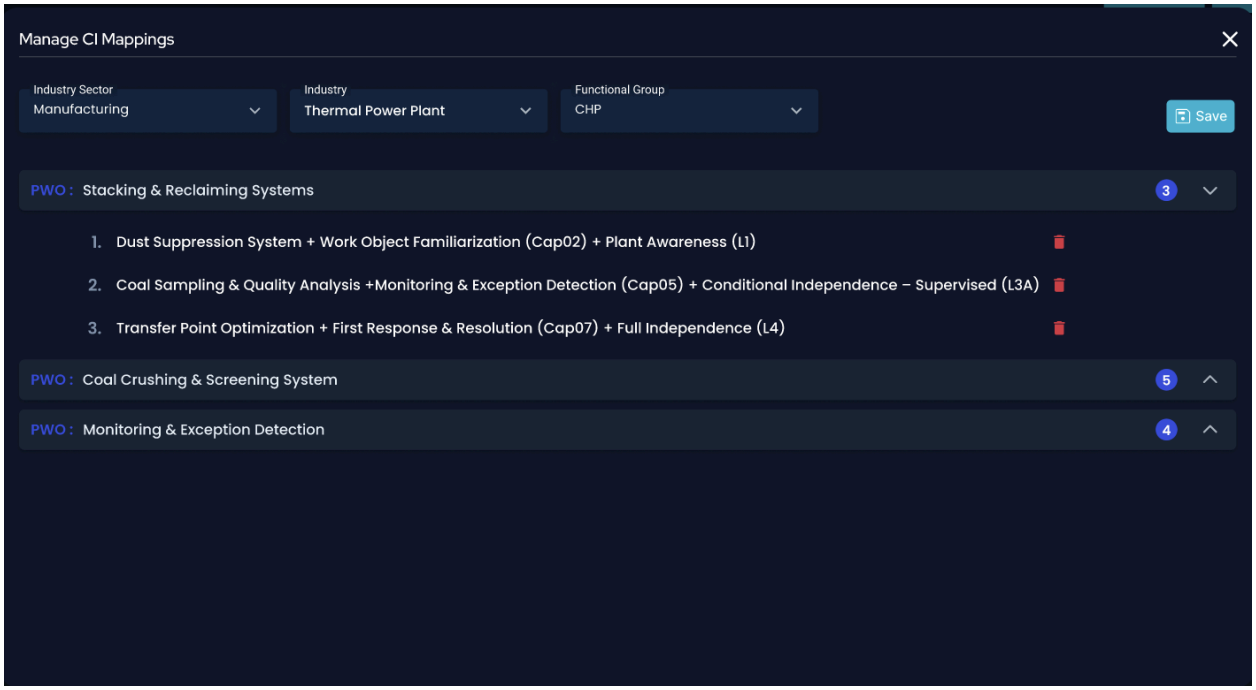
Picture 6: Publish Draft WRCF



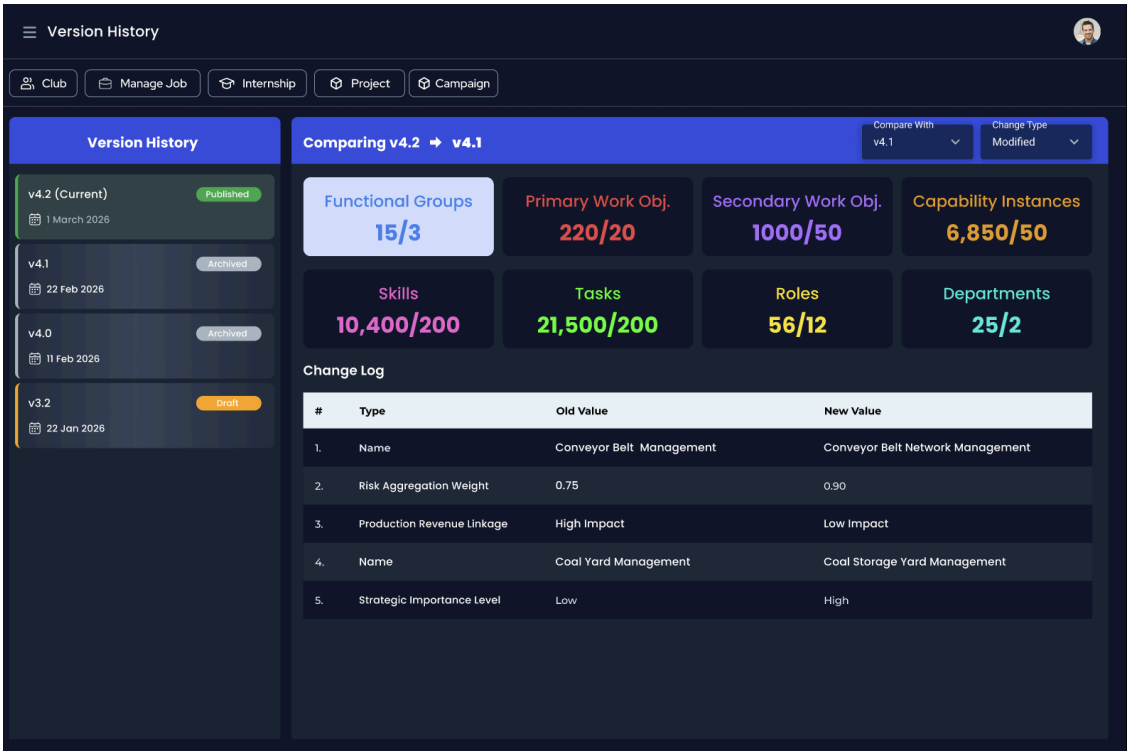
Picture 7: Manage WRCF Roles



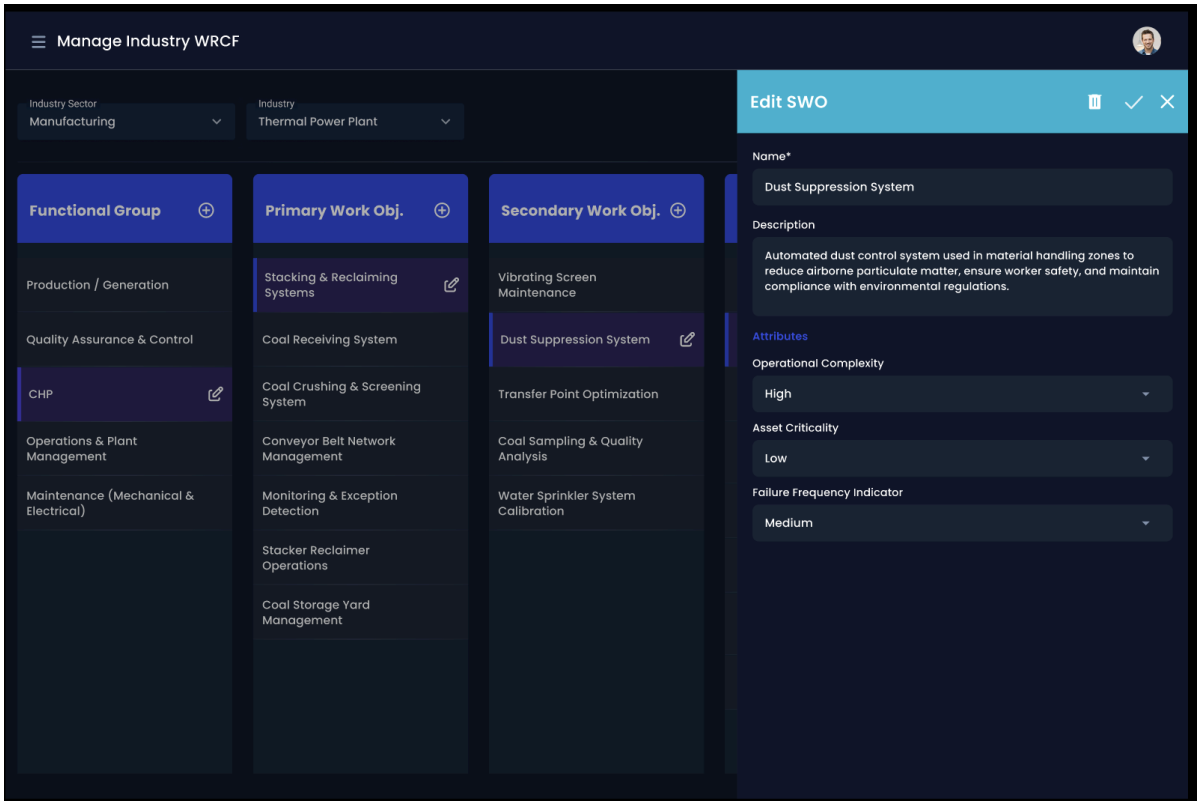
Edit Skills



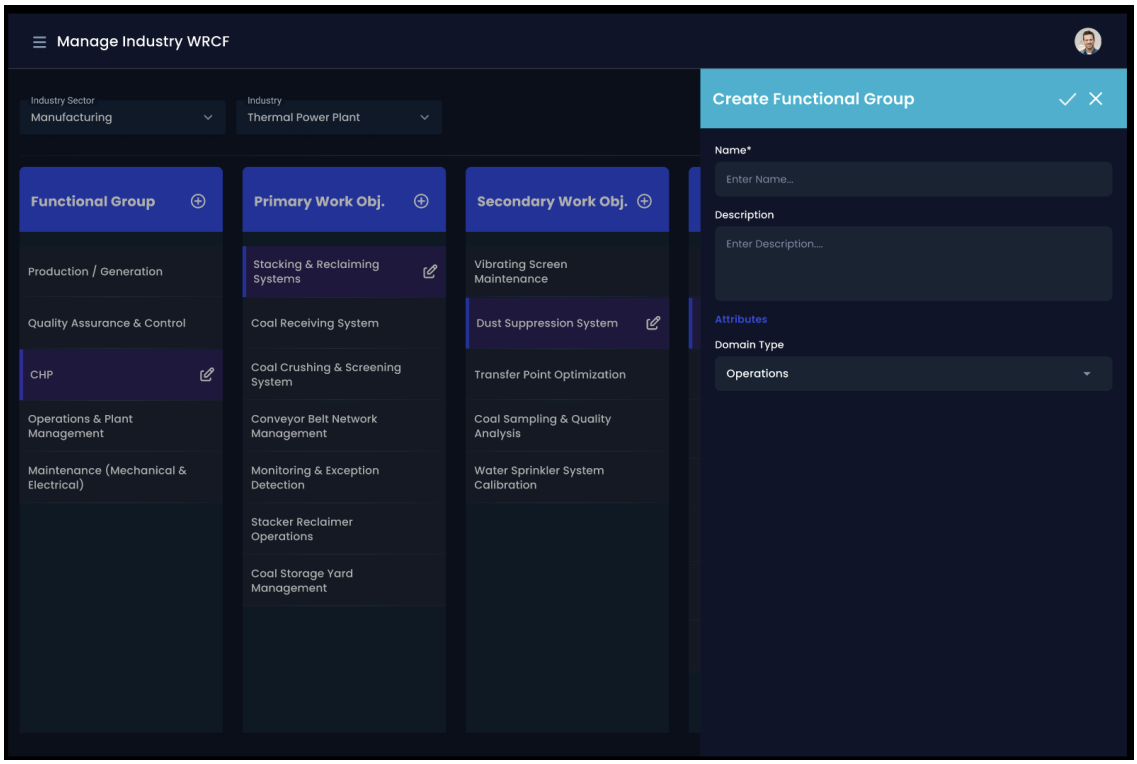
Picture 8: Manage CI Mapping



Picture 9: WRCF Version History



Edit SWO



Create Functional Group

```
Industry Sector
 → Industry
   → Functional Group
     → Primary Work Object (PWO)
       → Secondary Work Object (SWO)
         → Capability (Universal)
         → Proficiency (Universal)
           → Capability Instance (SWO + Capability + Proficiency)
             → Skill
               → Task
                 → Control Point
                   → Evidence
```

WRCF Hierarchy

1. Purpose and Scope

This document converts the visible WRCF screen behavior and the stated CRUD scope into a practical functional use case specification with detailed user stories.

The draft covers maintenance of industry groups, industries, function groups, primary work objects, secondary work objects, capabilities, proficiency levels, capability instances, skills, and visible template actions.

It translates:

- UI screens (10 screens provided)
- System hierarchy (as per screen 10)
- CRUD operations
- User roles

Back-end integration rules, approval workflow, detailed authorization matrices, and database-level design are intentionally left outside this document and should be refined separately.

WRCF module enables:

- Industry structure creation
- Capability architecture definition
- Capability instance mapping
- Skill-task-control hierarchy creation
- Role-based competency definition
- Versioning and publishing

2. Functional Interpretation of the Screens

- The page behaves as a hierarchical taxonomy maintenance workspace.
- Filters at the top progressively narrow context: Industry Group/Sector → Industry → Function Group.
- Vertical list panels expose the child entities and reference data maintained in the chosen context.
- Plus icons indicate create actions; pencil icons indicate edit actions; delete is assumed from edit or contextual action.
- The side drawer labeled 'Create SWO' shows the likely standard create/edit pattern for entity maintenance.
- Template buttons imply bulk maintenance support such as download, upload, or both.

3. Assumptions Used in This Draft

- Every child entity is dependent on its selected parent context.
- Ten universal capabilities and five proficiency levels exist as reference sets available to the work-object hierarchy.
- A capability instance is treated as a composite aggregate binding context, one capability, and one proficiency level.
- Skills are modeled as children of the capability instance because that is the most functionally useful interpretation from the screen.
- Delete should be dependency-aware and blocked where child records or linked mappings still exist.

4. Actors

Actor	Description	Primary Responsibilities
Whizard Super Admin	Owns reference taxonomy governance.	Manages Industry Sectors Manages Industries
Club Admin Club Industry Expert	Maintains domain structure in assigned contexts.	Creates and manages WRCF Defines WRCF structure Creates WRCF Industry roles Publishes WRCF versions
Viewer	Inspects hierarchy and mappings.	Searches, filters, and views WRCF structure and role data; no destructive actions.

5. Core Entity Hierarchy

Level	Entity	Parent	Notes
1	Industry Group / Sector	Root	Top-level business classification.
2	Industry	Industry Group	Example shown: Thermal Power Plant.
3	Function Group	Industry	Example shown: CHP.

4	Primary Work Object (PWO)	Function Group	Major work domain or object class.
5	Secondary Work Object (SWO)	Primary Work Object	More specific operating or maintenance object.
6	Capability	Reference set / context assignment	Universal capabilities
7	Proficiency Level	Reference set	Five standard levels including L3A and L3B.
8	Capability Instance	Composite aggregate	Context + capability + proficiency combination.
9	Skill	Capability Instance	Ability that enables a person to perform a defined action
10	Task	Skill	Executable work action performed on a specific SWO. Skills are validated through tasks.
11	Control Point	Task	Validation checkpoint
12	Evidence	Control Point	Proof of task execution
13	Evaluator	Control Point	Authorized supervisor validating evidence

Core Design Principles

1. **Functional Groups define where work happens.**
2. **Work Objects (PWO's & SWO's) are the anchor**
3. **Capabilities are universal**
4. **Proficiency defines autonomy**
5. **Skills are atomic abilities**
6. **Tasks are actions which validates skills**
7. **Control points define trust**
8. **Courses prepare; internships validate**
9. **Roles declare Competency**

FUNCTIONAL MODULES

🔷 MODULE 1: INDUSTRY WRCF DASHBOARD (Screen 1)

Use Case: UC-01 View WRCF Dashboard

Actor

Club Admin / Club Industry Expert

Description

User views high-level WRCF metrics for selected industry.

Precondition

- Industry exists
- WRCF initialized

Flow

1. User selects:
    - Industry Sector
    - Industry
2. System loads Values for:
    - Current Version
    - Draft Version
    - Last Updated
    - Validation Status
3. System displays counts:
    - Functional Groups
    - PWOs
    - SWOs
    - Capability Instances
    - Skills
    - Tasks
    - Roles
    - Departments

Actions

- Edit Structure
- Manage Roles
- Version History
- Publish Draft

User Stories

- As a user, I want to see WRCF summary metrics so that I understand completeness.
- As a user, I want quick actions so that I can navigate efficiently.

🔷 MODULE 2: STRUCTURE MANAGEMENT (Screen 2 & 3)

Use Case: UC-02 Manage Industry WRCF Structure

Description

User creates and manages hierarchical structure.

Main flow

1. User opens Manage Industry WRCF screen.
2. System loads available Industry Groups, Industries, and Function Groups.
3. User selects an Industry Group.
4. System filters Industries for the selected group.
5. User selects an Industry.
6. System loads FG, PWO, SWO, capabilities, proficiency levels for the selected context.

UI Behavior

5 Columns:

1. Functional Group
2. PWO
3. SWO
4. Capability (read-only)
5. Proficiency (read-only)

Core Rule

- Selected filter values persist during the session.
- Only valid child values are shown after each parent selection.
- Existing list panels refresh without full page reload where technically feasible
- Only ONE selection active per column

Capability is universal and industry agnostic. It is a reusable human action ability that can be trained and developed independent of role or context

Proficiency is the degree of autonomy, judgment, and responsibility with which a capability is exercised.

Sub Use Cases

UC-02.1 Create Functional Group

- Click "+"
- Open form
- Enter:
  - Name
  - Description
  - Attributes

- Save → appears in list

- System validates uniqueness within the function group, persists the record, closes the drawer, and refreshes the list.
- User may open an existing record via edit icon to update details or trigger delete if allowed.
- Delete is blocked if SWOs or capability instances depend on the PWO.

UC-02.2 Create PWO

- Requires Functional Group selected
- Same behavior as FG

UC-02.3 Create SWO

- Requires PWO selected
- Opens side panel (Screen 3)
- Includes attributes:
  - ✔ Operational Complexity (Low / Medium / High)
  - ✔ Asset Criticality (Low / Medium / High)
  - ✔ Failure Frequency Indicator (Low / Medium / High)

UC-02.4 Edit FG/PWO/SWO

- Select specific FG/PWO/SWO → edit icon appears
- Click on Edit icon → Opens edit panel
- Update fields (Name, Desc and Attributes)

User Stories

- As a user, I want hierarchical creation so that structure is logically built.

- As a user, I want contextual editing so that I don’t lose navigation.

🔷 MODULE 3: CAPABILITY INSTANCE CREATION (Screen 2 + 4)

Use Case: UC-03.1 Create Capability Instance (CI)

Description

Create mapping of:

SWO + Capability + Proficiency

Flow

1. Select:
    - Functional Group
    - PWO
    - SWO
    - Capability
    - Proficiency
2. Click ✔ (tick) on the Proficiency
3. System:
    - Adds selected combination to mapping akka "cart"

Behavior

- Tick turns from grey to green when selected
- User can then select another combination and add to the Mappings. Hence multiple CI selections allowed from same screen.
- Stored temporarily in mapping

Use Case: UC-03.2 Mapping View (Screen 4)

- Grouped by PWO
- Shows Capability Instances:

SWO + Capability + Proficiency

- Delete option available

Use Case: UC-03.3: Template Actions for bulk upload / download

- Download Template (FG/PWO/SWO)
- Upload Template

User Stories

- As a user, I want guided mapping so I don’t create invalid combinations.
- As a user, I want to view and manage all the selected CI together before saving
- As a user, I want batch upload so I can scale quickly.

🔷 MODULE 4: SKILL-TASK-CONTROL MANAGEMENT (Screen 5)

Use Case: UC-04 Manage Skills

Description

Define execution layer under Capability Instance.

Flow (to reach to this screen)

1. User Clicks on the 'Add Skills' button in the Industry WRCF Screen, then user will reach WRCF-Skill Screen

Layout

3 Columns:

- Skills
- Tasks
- Control Points

Right Panel: Create/Edit Form

Sub Use Cases

UC-04.1 Create Skill

Fields:

- Name
- Description
- Attributes
  - Cognitive Type
  - Volatility Index
  - Dependencies
  - Recertification Cycle

UC-04.2 Create Task

Fields:

- Name
- Description
- Attributes:
  - Execution Frequency
  - Variability
  - Duration
  - Required Skill Level

UC-04.3 Create Control Point

Fields:

- Name
- Description
- Attributes:
    - Risk Level
    - KPI Threshold
    - Failure Impact
    - Escalation Required
- Evidence
- Evaluator

Flow -Skill/Task/Control Point (Add / Edit)

1. User selects a capability instance or equivalent context row.

2. Select:
    - Skill
    - Task
    - Control Point

3. User clicks add icon in Skills panel.

4. System opens a skill form.

5. User enters skill name and optional description/metadata.

6. User saves.

7. System displays the skill in the Skills list and makes it editable.

Flow – Add Skill TaskControl Point to CI

8. Select Skill, then Task and Control Point

9. Click ✔ (tick) on the Control Point

10. System:

    - Adds selected combination to CI

Business rules / acceptance notes

- Skill name is mandatory.
- Duplicate skill names under the same capability instance are not allowed.
- Deleting a skill removes only the skill, not the parent capability instance.
- Delete of Skill is blocked if Task or Control Point is dependent on the Skill.

Behavior

- Tick turns from grey to green when selected
- User can then select another combination. Hence Control Points can be selected for the same CI
- Combination of Skill TaskControl point shall be unique
- Selection-based editing

- Edit panel context-sensitive
- Only selected item editable

User Stories

- As a user, I want a structured skill definition so that execution is measurable.
- As a user, I want control points so that validation is objective.

🔷 MODULE 5: VERSION MANAGEMENT (Screen 6 & 9)

Use Case: UC-05 Publish WRCF

Flow

1. Click "Publish Draft"
2. System shows warning:
    - Irreversible action
3. Confirm

System:

- Creates new version
- Archives old version

Use Case: UC-06 View Version History

Screen 9 Behavior

- Version list (left panel)
- Comparison panel (right)
- Shows:
    - Counts diff
    - Change log

User Stories

- As a user, I want version control so that changes are auditable.
- As a user, I want a comparison view so that impact is visible.

🔷 MODULE 6: ROLE MANAGEMENT (Screen 7 & 8)

Use Case: UC-07 Manage Roles

Structure

Industry
→ Department
→ Role
→ Capability Instances

Department is an Operational organizational unit &
Can map to multiple Functional Groups

Example:
Maintenance Department
→ Functional Groups: Boilers, Turbines etc

Role

Belongs to a Department
Inherits context from Department's Functional Groups
Has Multiple Capability Instances attached

Flow

1. Select:
    - Industry
    - Department
    - Role
2. If not exists:
    - Create Department
    - Create Role

Capability Assignment

4 Columns:

- PWO
- SWO
- Capability
- Proficiency

(All derived from Capability Instances)

Behavior

- Tick = add mapping
- Mapping grouped by PWO
- Delete option available

Save

- Saves Role → CapabilityInstance mapping

User Stories

- As a user, I want roles linked to capabilities so that competency is measurable.
- As a user, I want reuse of capability instances so that duplication is avoided.

🔷 MODULE 7: TEMPLATE MANAGEMENT

Use Case: UC-08 Import/Export Templates

Features

- Download templates
- Upload bulk data

Scope

- Functional Groups
- PWOs
- SWOs
- Skill
- Task
- Control Point

Flow:

1. User clicks a Template action button.
2. System offers either a downloadable template, an upload action, or both depending on permissions.
3. For upload, user selects a file.
4. System validates schema and reports row-level errors if present.
5. Successful import updates the corresponding lists.

Business rules / acceptance notes

- Template format must match expected entity structure.
- Import must not partially corrupt existing data; errors should be transactional or clearly reported.
- Downloaded templates should reflect current mandatory columns and controlled values.

- System should log the No of records uploaded or failed. Partial record update should not be allowed.

User Stories

- As a user, I want to do bulk upload so that large WRCFs can be created efficiently, so I should be able to fetch the right template for uploading the bulk data.
- As a user, I want bulk upload so that large WRCFs can be created efficiently. so I should be able to populate the downloaded template and then upload the bulk data.

🔷 MODULE 8: Industry Management

UC-09

As a Whizard administrator, I want to filter by Industry Group, Industry, and Function Group so that I work only on the relevant taxonomy context.

Acceptance criteria

- When I change a parent filter, child filters are refreshed.
- No orphan values are shown.
- The visible lists correspond to the active context.

UC-10

As a Whizard Administrator, I want to create a new Industry Group so that new domains can be added to the model.

Acceptance criteria

- I can open a create form from the group context.
- Name is mandatory.
- The system rejects duplicate names.

UC-11

As a Whizard Administrator, I want to create and edit Industries under a selected group so that the structure reflects the business domain.

Acceptance criteria

- Industry cannot be saved without a selected group.
- Edits are shown immediately in the industry list.
- Deletion is blocked when function groups exist.

UC-12

As an administrator, I want delete operations to be dependency-aware so that reference integrity is preserved.

Acceptance criteria

- System shows a clear validation message when delete is not allowed.
- The message indicates what child or linked data blocks deletion.
- No silent data loss occurs.

UC-13

As an administrator, I want to download a maintenance template so that I can prepare bulk updates offline.

Acceptance criteria

- The template includes the right columns.
- Mandatory fields are clearly indicated.
- Lookup columns use business-friendly labels.

UC-14

As an administrator, I want to upload a populated template so that large data sets can be loaded quickly.

Acceptance criteria

- The system validates file structure.
- Errors are shown with enough detail to correct the file.
- Successful imports refresh the UI lists.

9. CRUD Coverage Matrix

Entity	Create	Read/View	Update	Delete
Industry Group	Yes	Yes	Yes	Yes, if no child industries
Industry	Yes	Yes	Yes	Yes, if no function groups
Function Group	Yes	Yes	Yes	Yes, if no PWOs
Primary Work Object	Yes	Yes	Yes	Yes, if no SWOs/dependent instances
Secondary Work Object	Yes	Yes	Yes	Yes, if no dependent capability instances
Capability	Yes	Yes	Yes	Yes, if not in use
Proficiency Level	Yes	Yes	Yes	Restricted; only if not in use
Capability Instance	Yes	Yes	Yes	Yes, subject to linked skills/dependencies
Skill	Yes	Yes	Yes	Yes

10. Validation and Business Rule Notes

- Names should be unique within their immediate parent scope.
- Mandatory fields should be visually marked in create/edit drawers.
- Controlled values should be used for dropdown attributes rather than free text where business classification is intended.
- Delete should require confirmation and should explain dependency blockers.
- The UI should preserve filter context after create, update, import, or cancel actions.
- Any template import should validate parent-child references before committing records.
- Capability instance creation should prevent duplicate combinations for the same hierarchy scope.
- Audit fields such as created by, updated by, and updated date are recommended even though they are not visible in the screenshots.

11. Open Points for Business Confirmation

- Whether delete should be hard delete, soft delete, or archive/inactivate.
- Whether separate role-based permissions exist for create, edit, delete, and import.